# AgentSLA: A Framework for AI Agent Service Level Agreements

## Executive Summary

The rapid transition from a **Model-as-a-Service** paradigm to **Agent-as-a-Service** has introduced significant challenges in ensuring software quality. While traditional software systems rely on Service Level Agreements (SLAs) to guarantee Quality of Service (QoS), existing formalisms fail to account for the unique characteristics of AI agents, such as autonomy, non-deterministic behavior, and performance drift.

This document outlines the development of **AgentSLA**, a comprehensive framework designed to bridge this gap. The framework consists of two primary components:

1. **An Extended Quality Model:** An evolution of the ISO/IEC 25010 standard, modified to include dimensions critical to AI, such as sustainability, autonomy, and output properties.
2. **The AgentSLA Domain-Specific Language (DSL):** A JSON-based language that enables providers and consumers to define, exchange, and automatically process formal agreements tailored to agentic systems.

Key takeaways include the introduction of specialized metrics to track **QoS Drift**, the integration of **Model Cards** into service scopes, and the provision of a Python-based validating parser to support automated monitoring and enforcement.

--------------------------------------------------------------------------------

## The AI Agent Quality Model

To establish effective SLAs, there must first be an agreed-upon model for what constitutes "quality" in an AI agent. The proposed model extends the **ISO/IEC 25010** standard, adding specific characteristics relevant to the agentic paradigm.

### Primary Extensions to ISO/IEC 25010

The framework identifies nine existing sections and introduces two entirely new sections to capture the full scope of AI performance.

| Category | Extensions and Subcharacteristics |
| --- | --- |
| **Sustainability** (New) | Training impact, inference impact, and mitigation efforts (e.g., carbon offsets). |
| **Output Properties** (New) | Conciseness, consistency, creativity, and diversity of generated results. |
| **Interaction Capability** | Added **Autonomy**, representing the level of human oversight vs. independent agent refinement. |
| **Reliability** | Added **Fairness** to address ethical biases (racism, sexism, etc.). |
| **Security** | Extended **Accountability** to include traceability of actions through the AI model. |
| **Functional Suitability** | Broadened to include **Functional Completeness** (capabilities), **Correctness** (accuracy), and **Appropriateness** (relevance). |
| **Performance Efficiency** | Extended **Time-behavior** to include Time-To-First-Token (TTFT) and model training time. |

## Core Quality Metrics

The framework provides a non-exhaustive set of metrics to assess these characteristics. These metrics are categorized by their parent in the quality model:

- **Accuracy & Correctness:** Precision, Recall, F1 Score, and AUC (Area Under Curve).
- **Robustness & Fault Tolerance:** Pointwise Robustness, Adversarial Frequency, and Adversarial Distance.
- **Interpretability:** SHAP (SHAP estimation error) and LIME (local surrogates comparison).

- **Sustainability:** Estimated energy consumption, water consumption, and carbon emissions.
- **Agent-Specific:** Oversight Level (Autonomy), Output Size (Conciseness), and protocol compatibility (A2A, MCP).

--------------------------------------------------------------------------------

# AgentSLA: Domain-Specific Language (DSL)

AgentSLA is a formal language designed to model agreements between service providers and consumers. Its concrete syntax is based on **JSON** to ensure compatibility with emerging agent protocols like Google's **Agent2Agent (A2A)**.

## Abstract Syntax and Metamodel

The structure of an AgentSLA agreement is built on several key classes:

- **SLA:** The root element containing one or more Guarantee Terms.
- **GuaranteeTerm:** Defines the obligations, consisting of a Scope (the specific agent/model), Qualifying Conditions (pre-conditions), and Service-Level Objectives (SLOs).
- **SLO:** Specifies a QoS condition using boolean expressions and comparisons (e.g., "TTFT must be LESS than 1 second").
- **QoSMetric:** Includes attributes for name, description, unit, and **uncertainty**. The uncertainty value accounts for the non-deterministic nature of AI measurements.

## Specialized Metric Types

AgentSLA introduces two advanced metric types to handle the temporal complexities of AI agents:

1. **DerivedQoSMetric:** Aggregates measurements over a specific window (e.g., the average TTFT across the last 10 messages).
2. **QoSDriftMetric:** Evaluates the stability of an agent by comparing two derived metrics over time. This detects whether performance is enhancing or degrading as the model context evolves.

--------------------------------------------------------------------------------

# Tooling and Implementation

The framework includes a Python-based implementation to facilitate the adoption of AgentSLA in real-world systems.

- **Metamodel Generation:** The abstract syntax was modeled using the **BESSER Low-Code platform**, which generated the underlying Python class definitions.
- **Validating Parser:** A tool that transforms JSON descriptions into AgentSLA model instances. It performs several validation checks:
    - Ensuring required units are present and conform to the metamodel.
    - Verifying that operators and aggregation functions (MIN, MAX, AVG, MED) are valid.
    - Confirming that provider confidence values fall within the [0, 1] interval.
- **Model Card Integration:** The DSL allows agents to refer to their specific AI models via **Model Cards**, which report on intended use, data, performance, and ethical considerations.

--------------------------------------------------------------------------------

# Strategic Applications and Future Directions

The formalization of AgentSLAs enables several automated scenarios for smart software systems:

- **Automated Agent Selection:** Systems can dynamically choose between agents based on geographic location (EU vs. US), consumption metrics, or interoperability with local tools.
- **Dynamic Monitoring and Orchestration:** AgentSLA models can be used to automatically deploy monitors. If an agent fails to meet its SLOs, an orchestrator can dynamically switch to a different provider to enforce the agreement.
- **Protocol Integration:** Future work aims to include AgentSLA directly within the A2A protocol and provide support within Agent Development Kits (ADKs).
- **User Accessibility:** Plans are underway to develop graphical notations and conversational interfaces for users who prefer non-textual ways to define service requirements.

As AI software continues to evolve, the AgentSLA framework serves as a foundational step toward establishing rigorous quality assurance and formal contractual standards for the Agent-as-a-Service era.